Technical Challenge Summary

Candidate: Pammela Vicenti Ribeiro
Role: AI Engineer - Full-Stack for Generative AI Applications
Project: Empório da Música AI Support Assistant

Objective

This project prototypes a conversational customer support assistant for Empório da Música, a fictional musical instrument store. The assistant answers store-policy questions and consults the supplied operational data for products, prices, promotions, stock, and order status.

Solution

The solution is a Python command-line application connected to the Gemini API through Google's official Gen AI SDK. The corrected catalog run used `gemini-2.5-flash`; unchanged scenarios 02-04 used `gemini-3.5-flash-lite`. The model manages conversation and selects read-only functions. Python performs CSV queries, privacy checks, and BRL calculations with `Decimal`. A reviewed extraction of the short policy manual supports lexical retrieval without a vector database.

The main tools are product search, exact price calculation, privacy-limited order lookup, and policy retrieval. Search supports pagination with total/returned counts and a next offset, plus structured specification filters applied before pagination. Installment schedules validate their count and adjust the last installment so rounded values sum to the final price. Strict JSON schemas constrain tool inputs. CSV content, retrieved text, and user messages cannot override the assistant's instructions.

Reliability and safety

The implementation does not invent a policy when a source is missing or contradictory. It explicitly handles two conflicting WhatsApp numbers and inconsistent minimum credit installments in the manual. It also treats positive-stock products marked `coming_soon` or `discontinued` as unavailable and does not infer a return deadline from an order date when the actual receipt date is absent.

Order status requires the order ID and exact registered e-mail. Successful results omit the customer's name, phone, e-mail, and city. This is deliberately documented as prototype verification rather than production authentication.

Configuration and execution

The application reads `GEMINI_API_KEY`, `GEMINI_MODEL`, and an optional `EMPORIO_DATA_DIR` from `.env`. Secrets are ignored by Git and excluded from review packages. The model-access probe lists models visible to the configured key/project and forces a Gemini function call before a model is accepted.

The principal commands are `python -m pip install -e .`, `python scripts/check_model_access.py`, `python -m emporio_agent`, `python scripts/generate_examples.py`, and `python -m unittest discover -s tests -v`. The generator also supports `--resume` and `--scenario` for interrupted or targeted live runs.

Validation and real results

Thirty-nine local tests pass on Python 3.13.1. They cover source loading, configuration, specifications, filters, availability, promotions/PIX, exact installments, policy conflicts, privacy, retrieval, tool failures, model-access validation, and the mocked Gemini function-calling loop. Six new regressions cover exact GN-15 cents, Shelby 107 beyond the initial eight, specification filtering across the catalog, pagination bounds/metadata, and tool-dispatch support.

The previous catalog approval is revoked: its second answer changed GN-15 from BRL 689.00 to 689.90 and falsely generalized the lack of electronics from eight results to the whole budget range. That output is retained unchanged under `examples/rejected/`.

Scenario 01 was regenerated on 2026-11-05 at 17:52 America/Sao_Paulo with real Gemini, after 503 errors on the original model. It preserves GN-15 at BRL 689.00 across turns and limits the electronics statement to the eight shown products. An added catalog-wide question triggers an `electronics=yes` search and correctly finds Shelby SGD-195E (107) at BRL 799.00. All nine returned product records were checked directly against the CSVs; the real tool trace is retained. The reviewed transcript is byte-identical to generated output. Scenarios 02-04 retain their prior source reviews and were not rerun. No assistant response was manually rewritten. The public repository was prepared with the candidate's authorization; no e-mail or form submission has been sent.

Limitations and next steps

The CLI keeps history only during the process; policy search is lexical and the CSV snapshot is not transactional. Four curated scenarios are not a statistical benchmark. Price copying and scope instructions still require live source review; tool tests cannot guarantee prompt compliance. External capacity and quotas affect execution. The prototype cannot modify orders. Production needs strong authentication, a live backend, rate limiting, privacy-aware telemetry, policy governance, and broader evaluations.

Use of AI assistance

OpenAI Codex assisted with source inspection, architecture, implementation, tests, and documentation. Source facts were verified against the original PDFs and CSVs, and the candidate is responsible for reviewing, understanding, and presenting the final work.

Repository URL: https://github.com/pammelavicenti/emporio-musica-ai-agent